



BACHELOR'S THESIS
(COURSE CODE: XB_40001)

Web-Based System for Transparent Analysis of AI Coding Assistant Utilization

by

Pham Ngoc Diep

(STUDENT NUMBER: 2807668)

*Submitted in partial fulfillment of the requirements
for the degree of
Bachelor of Science
in
Computer Science
at the
Vrije Universiteit Amsterdam*

July 7, 2026

Certified by

First supervisor's name
First supervisor's title
First Supervisor

Certified by

Daily supervisor's name
Daily supervisor's title
Daily Supervisor

Certified by

Second reader's name
Second reader's title
Second Reader

Web-Based System for Transparent Analysis of AI Coding Assistant Utilization

Pham Ngoc Diep
Vrije Universiteit Amsterdam
Amsterdam, NL
n.d.pham@student.vu.nl

ABSTRACT

GitHub Copilot and similar AI coding assistants are now part of a software developer's daily work. Since Copilot moved to usage-based billing, costs vary per user and per AI model, which makes it important to view that usage in a structured way. GitHub provides detailed usage logs by default, but the full picture is only reachable through an enterprise account, which not everyone has. At Van Lanschot Kempen this means the data reaches the management team as spreadsheets, exported manually by one person. Management sees rough totals and developers have almost no view of their own usage. The goal of this project is to make AI coding assistant utilization analyzable and transparent for both groups. I design and build a web-based analytics dashboard that transforms raw Copilot usage logs into interactive views, with filtering and grouping across time, user, AI model, team, and department. The outcome is a working application and a structured approach for analyzing AI coding assistant usage and cost data in an organisational setting, intended to be used across employees in different roles in the company.

1 INTRODUCTION

AI coding assistants have become a normal part of software development. GitHub Copilot alone had around 4.7 million paid subscribers as of January 2026, and GitHub reports it is used by roughly 90% of the Fortune 100 [7]. Peng et al. [18] ran a controlled experiment with 95 professional developers and found that those with Copilot access completed a programming task 55.8% faster than the control group, which goes a long way to explaining why.

A free tier exists, but it comes with strict request limits and access only to lighter models. Organisations that want the full capability, including access to more powerful models and organisational usage reporting, need a Business or Enterprise plan [11].

Van Lanschot Kempen is one such organisation [19]. It is a Dutch financial institution subject to EU financial regulation, including the Digital Operational Resilience Act [5], which sets binding requirements for ICT risk management across the software lifecycle. Developers in its Digital Innovation & Analytics (DIA) department use Github Copilot daily. Recently the cost of that adoption changed. On June 1, 2026, GitHub switched Github Copilot from a per-seat subscription to an AI credit system, where different models consume credits at different rates [10]. The bill now depends on who used which model, how often, and for what. Costs vary per user, per model, per day.

The data to understand this exists. GitHub records every usage event with the user, the date, the model, and the credit and cost amounts. The problem is what happens to that data afterwards. At Van Lanschot Kempen, one person downloads the billing export by hand and turns it into a spreadsheet. Management receives a

large, complex export that is hard to extract insight from directly. Developers can see how many credits a single request consumed in their IDE, but they have no view of what that translates to in money, or how their usage compares to colleagues. There is a further complication: the billing export only knows GitHub usernames. It has no record of which team or department a user belongs to. That link comes from a separate company database, so anyone who wants to break costs down by team or department has to combine the two sources themselves and do the calculation by hand. On top of that, the Copilot usage metrics API only keeps a rolling 28-day window of data [1]. Anything older than that is gone unless someone has already exported and stored it. The information is technically there and practically inaccessible. That gap is what this thesis means by a lack of transparency.

There is a second reason to care about usage patterns, beyond the bill. Brandebusemeyer et al. [2] followed 22 professional developers at SAP and found that the way developers interact with Copilot changes its value: using either in-code suggestions or chat made them measurably faster, while combining both in one task brought no gain at all. How a tool is used matters as much as whether it is used. This project builds a web-based analytics dashboard that turns the raw billing data into interactive views. Users can filter by user, model, and time period, and group costs along organisational and technical dimensions: department, team, model, user, and month. The dashboard serves both audiences at once. Developers get a clearer picture of their own usage, what it costs in money, and how it sits relative to the rest of the team. Management gets the ability to explore instead of receiving a static export and having to interpret it by hand.

The main research question is:

How can utilization of AI coding assistants be systematically analyzed through a web-based analytics dashboard?

This breaks down into four sub-questions:

- **SQ1.** How can usage logs from AI coding assistants be structured into an analytical data model?
- **SQ2.** What visualization techniques are most appropriate for analyzing AI assistant usage logs?
- **SQ3.** How can a web-based dashboard be implemented and securely deployed to support analysis of AI coding assistant usage inside a corporate environment?
- **SQ4.** To what extent does a web-based analytics dashboard improve the interpretability and transparency of AI coding assistant utilization?

Section 2 covers the background on AI coding assistants and visual analytics. Section 3 explains the methodology. Sections 4 and 5 cover the design and implementation, Section 6 the evaluation.

2 BACKGROUND

Before the design decisions make sense, two things need context: how Copilot usage turns into cost data, and why interactive visualisation is the right way to analyse it. The section ends with a short overview of the technology the dashboard is built on.

2.1 AI Coding Assistants

AI coding assistants are tools that use large language models to support software development work. They integrate directly into code editors and respond to a developer's actions in real time. GitHub Copilot is one of the most widely used tools of this kind, with around 4.7 million paid subscribers as of January 2026 [7], and it is the one used at Van Lanschot Kempen.

Copilot offers several distinct interaction modes, each suited to a different kind of task [9]. Code completions are the foundational mode: ghost-text suggestions appear inline as the developer types, drawing on the surrounding code as context. These suggestions are fast and low-friction, and they are not billed in AI credits: they remain unlimited on all paid Copilot plans [12]. The chat interface is where the more expensive interactions happen. Within VS Code, developers can use three chat modes: Ask, Edit, and Agent. Ask is a conversational mode for questions and explanations, with the model considering the current file and any referenced context. Edit mode applies suggested changes across one or more files at once. Agent mode is the most autonomous of the three: it can plan a multi-step task, decide which files to touch, run terminal commands, and iterate until the goal is reached [8]. These modes vary significantly in how much context they pull in and how many tokens they consume per interaction.

None of these interaction modes are interchangeable, and different AI models are available behind them. Users can switch between models from providers like OpenAI, Anthropic, and Google, depending on what the plan allows. Each model has its own credit cost per token [12]. The combination of interaction mode and model choice determines what an interaction costs. This is why a usage log that captures who used which model and which mode, and how often, is also the data that explains the bill. Brandebusmeyer et al. [2] noted that different interaction types run through different prompt templates and differ in how much of the codebase they consider as context, which produces different model outputs. That observation maps directly onto the credit billing structure that GitHub introduced in June 2026, where cost follows token consumption.

2.2 Visual Analytics and Dashboards

Visual analytics is the combination of automated data analysis with interactive visualisation, aimed at supporting understanding and decision making on complex datasets [13]. Keim et al. frame the motivation as an information overload problem: raw data accumulates faster than anyone can process it manually, and the solution is not to hand everything to an algorithm either.

The reason both extremes fall short is worth spelling out. A fully automated analysis can summarise a dataset quickly, but if the result looks wrong there is no way to inspect what the algorithm did [13]. Manual inspection of a large or multi-dimensional dataset is the opposite problem: a person can reason well about what they see, but cannot get through the volume. Visual analytics puts a

human in the loop at the right moment. The system computes and summarises; the user looks at the result, questions it, and adjusts the view. Keim et al. call this the sense-making loop. The user does not just consume output, they steer the analysis based on what the visualisation reveals.

A dashboard is one concrete way to build this kind of system. It puts charts, tables, and filters in one place so the user can change what they are looking at without asking anyone or re-running anything. The interactivity is what matters. Keim et al. describe the appropriate design principle as: analyse first, show what is important, then let the user zoom, filter, and request details [13]. Without filtering and drill-down, a dashboard is just a chart on a screen, not much better than a printed summary.

For this project, the dataset is the credit billing records of GitHub Copilot across the organisation. It has five dimensions to group by (user, model, date, team, department), three cost measures (credits consumed, gross cost, net cost), and two audiences who ask different questions about it. A manager wants to know which team is spending the most. A developer wants to see their own usage in euros. A single flat export cannot answer both at the same time, and a single aggregate number loses the structure. That is the problem a dashboard with grouping and filtering is meant to solve.

2.3 Technology Stack

The dashboard is built as a web application, which keeps it accessible through a browser with no install step for anyone in the company. The deployment infrastructure, covering CI/CD pipelines, container security, and network isolation inside the company's Azure environment, is covered in Section 5.

The frontend uses Angular 20 [6], a TypeScript framework for building single-page applications. Angular 20 makes signals a central part of its model: a signal is a reactive value that other computed values can depend on, and when it changes, only the parts of the interface that depend on it update. This makes it practical to wire filtering, grouping, and chart data together so that adjusting a filter instantly re-derives everything downstream without any manual re-fetch. Charts are rendered with Chart.js, a widely used JavaScript charting library, integrated into Angular through the ng2-charts wrapper.

The backend is a .NET 9 Minimal API written in C#. It exposes the usage data as a REST endpoint and reads from a SQL Server database through Entity Framework Core, an object-relational mapper that lets the data model and queries be expressed in C# rather than raw SQL. The reasons behind each of these choices, and how they shaped the implementation decisions, are discussed in Sections 4 and 5.

3 METHODOLOGY

The project follows the engineering cycle described by Wieringa [20], a structured approach to design science research with four phases: problem investigation, treatment design, treatment validation, and treatment implementation. Since this dashboard was built and evaluated directly inside the organisation it is meant for, implementation and validation ran together in practice. What is learned during building and evaluation feeds back into earlier decisions, so the cycle does not run in one straight direction. Figure 1 shows

how the phases map onto this project and onto the sub-questions from Section 1.

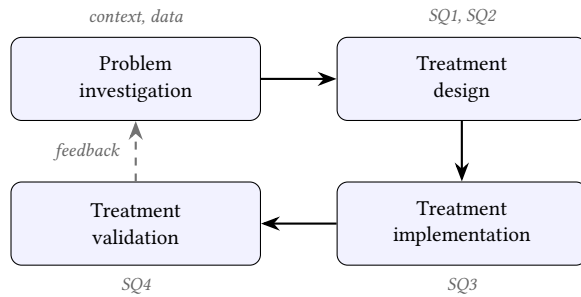


Figure 1: The engineering cycle applied to this project.

3.1 Problem Investigation

The starting point was understanding how Copilot reporting currently worked at Van Lanschot Kempen. I spoke with the software engineers closest to the process. The main stakeholder was an expert software engineer who managed the existing spreadsheet exports and held the enterprise account, which gave him direct access to the billing data. He had also been the main contact between the development team and management, so he understood what both sides needed from the data. A senior software engineer acted as mentor throughout the project. I looked at what the monthly exports actually contained, what questions people wanted to answer from them, and where the current process broke down.

I also studied literature on visual analytics [13] and on how developers use AI coding assistants [2, 18] to understand what existing research said about both the tool and the problem of making its data understandable. The raw data itself was part of the investigation too: GitHub provides billing exports in CSV and NDJSON formats, and the column structure determines what kinds of analysis are possible on top of it. One practical constraint that came up early is that the Copilot usage metrics API only exposes a rolling 28-day window [1]. Data older than that is not available through the API at all, which means any historical analysis depends on someone having already exported and stored it before it aged out.

During this phase, GitHub announced that Copilot would switch from per-seat billing to an AI credit system on June 1, 2026 [10], with different models consuming credits at different rates [12]. The export schema changed with it. This meant the data model I was working with shifted in the middle of the project, and some of the investigation work had to be repeated with the new format.

3.2 Structuring the Data (SQ1)

The data source for this project was not something I designed from scratch. The billing data comes from GitHub and arrives in a format GitHub defines. Initially, the plan was to work with CSV and NDJSON file exports: a team member would download the export from the GitHub portal and upload it through the application, which would parse and store it in a single flat database table. That approach was built and worked, but it still depended on one person performing the upload manually.

Later in the project, the main stakeholder set up an automated job that pulls data from multiple sources directly into the project's own database, without any manual step. At that point the data model was redesigned. Instead of one flat table that embedded the username directly, the new model uses two tables: a billing table with one row per user per day per model, and a separate team member table that maps GitHub usernames to organisational metadata like department and team. To answer SQ1, I analysed what the records measure (credits consumed, gross cost, net cost) and what dimensions someone would want to break those measures down by (user, AI model, date, department, team), and used that to define the structure. Section 4 covers the details.

3.3 Selecting Visualization Techniques (SQ2)

For each question the stakeholders actually asked, I looked at which chart types could answer it and picked the simplest one that worked. The main questions were: which model costs the most, how does usage change over time, and which users or teams consume the most credits. Keim et al. [13] argue that interactivity is what separates visual analytics from static reporting, so filtering, grouping, and drill-down were treated as requirements from the start. A chart that cannot be filtered by time period or model would just recreate the spreadsheet problem inside a browser.

3.4 Building the Application Securely (SQ3)

The dashboard was developed iteratively inside the company's own infrastructure, in two-week sprints. Three people were closely involved. The main stakeholder, an expert software engineer, set up and maintained the database and handled GitHub plan changes including the move to token-based usage. He provided ongoing feedback on the features. A senior software engineer acted as mentor, responsible for code review and code quality throughout. A second software engineering intern worked alongside me on usage-metrics features, such as how much developers use Copilot and which IDE they work in.

On the application side, development went through two phases: a manual file import approach in April and May 2026, and a direct database connection from June 2026 onwards. SQ3 also covers the deployment infrastructure, which I built from scratch. Van Lanschot Kempen is a financial institution and the data the application handles is tied to identifiable employees, so the deployment could not be a simple cloud push. I set up separate CI/CD pipelines in Azure DevOps for the frontend and backend. Each pipeline enforces security at multiple points in the build: the frontend checks production dependencies for known vulnerabilities before compilation; the backend validates the Bicep infrastructure templates against the real Azure subscription before any application code is compiled; the container image is scanned for CVEs and embedded secrets before it reaches the registry. The application itself runs inside the company's Azure virtual network with no public access to the API, and service-to-service authentication uses Azure Managed Identities so no credentials are stored anywhere in the pipeline or the image. These were not design choices made after the fact. They were requirements for running inside a regulated environment, and they are part of what answering SQ3 meant in this project.

3.5 Evaluating the Dashboard (SQ4)

The evaluation runs at two levels. Throughout development, each sprint ended with a review where I presented progress and stakeholders gave feedback. Both software engineers and a manager attended, which meant the same feature got looked at from the technical side and the management side in the same meeting. At the end of the project, three face-to-face interviews give a more considered view from people who represent the dashboard's actual users: a software engineer not involved in building it, the main stakeholder, and a manager from the management team. The setup and results are described in Section 6.

3.6 Constraints

The project ran inside a real organisation with real billing data tied to identifiable employees, so privacy was a constant consideration. The application only exposes data through joined views that show summed totals per grouping level, and the future potential role-based access control limits who can see whose records. The data source and schema were not choices I could make freely, since both are defined by GitHub and by the company's infrastructure. This is why some decisions, like redesigning the data model mid-project, were responses to external changes rather than planned design moves.

4 DESIGN

This section covers the design decisions behind the dashboard. The data model design answers SQ1 and the visualization choices answer SQ2, the two sub-questions from the treatment design phase in Section 3. The implementation follows in Section 5.

4.1 Requirements

The requirements came from the two user groups identified in Section 1. Management and finance need to know which AI models generate the most cost, how consumption develops over time, and which users or departments drive it. Developers already see how many credits a single interaction consumes in their IDE, but they have no view of what that translates to in money, or how their usage compares to others on the team. Both groups need to answer their questions themselves, without asking the one person who currently controls the exports.

From those questions I derived the requirements: the data must be filterable by user, model, and time period, groupable along organisational and technical dimensions, and visible both as exact numbers and as visual patterns. Keim et al. [13] make the point that interaction is what turns visualization into analysis, so filtering and grouping were treated as core requirements rather than optional extras. A static chart of monthly totals would just be the spreadsheet again, rendered in a browser.

4.2 Measures and Dimensions (SQ1)

The design separates what is measured from how it can be broken down. The measures are the credit quantity, the gross amount, and the net amount of each usage event. The dimensions are the user, the AI model, the date, and the organisational placement of the user: department and team.

This separation maps directly onto the two-table data model. A billing table holds one row per user per day per model with the three measures. A separate team member table maps a license identifier to a display name, a department, and a team. The organisational hierarchy is deliberately not stored in the billing rows. When someone switches teams, their billing history stays untouched, and the link is resolved through a join at query time.

The split has a second effect that fits the transparency goal: the frontend never receives raw license identifiers, only readable names and summed totals.

4.3 Choosing Visualization Techniques (SQ2)

I picked the chart types from the questions the users ask, not from a list of options. Comparing cost across models is a bar chart. Usage over time is a line chart, with two y-axes because credits and money are different scales. Ranking the heaviest users is a horizontal bar chart with a top-N cutoff.

Charts are good for seeing patterns and comparing across a dimension at a glance, but they do not show exact numbers well. You cannot read one developer's cost in one specific month off a bar. The table covers that side, with precise credit, gross, and net amounts and a drill-down structure that lets a manager navigate from department level all the way to an individual's daily record. So the design ended up as two complementary views over the same data: charts for the overall picture and trends, the table for the detail behind them.

Keim et al. [13] mention richer techniques like treemaps for hierarchical data, but a collapsible table maps more directly onto the grouping levels the users already think in, and is easier to read when the numbers matter as much as the shape. Whenever a simpler option answered the question, I went with it.

Both views share the same endpoint and the same filters, so a question raised in one can be checked in the other.

4.4 Grouping Design

The table groups records along up to five levels: department, team, model, username, and month. The user decides which levels are active and in which order. That ordering flexibility is where most of the value sits. Grouping department before model answers "what does each department spend per model", while the reverse answers "which departments drive each model's cost". Both are legitimate questions, so neither ordering should be hardcoded.

Every group node already shows the totals for credits, gross, and net without needing to open it. Expanding a branch is for finding out *why* a number is what it is, not for learning the sum.

4.5 Filtering Design

Filtering happens at two layers. The API accepts a partial username, an exact model name, and a date range, and applies them in the database query so a client only receives the slice it needs. The frontend holds its own filter state on top of that, which applies instantly to the already-loaded data without a round-trip. The table follows the global filter; each chart additionally has its own independent controls, because narrowing to one model for a trend chart should not force the whole page to that model.

Both the filter dialog and the group-by dialog follow a reversible edit design: changes are staged in a temporary copy and only committed on apply. Cancelling throws the copy away, so a half-finished edit never leaks into the live view.

4.6 Access Control Design (Future Potential Feature)

Authentication and authorisation are fully designed and planned for the next development phase. The design uses Microsoft single sign-on through Azure AD, since every intended user already has a Microsoft work account. Four roles control what a signed-in user can see: an individual contributor sees only their own usage, a team manager sees their team, a department manager sees their department, and admin or finance roles see everything. Role and scope assignments live in the application's own database instead of Azure AD groups, which keeps role changes inside the application and avoids needing directory admin rights for every update. Scoping is enforced server-side from the token claims, so a hand-crafted API request still returns only authorised data. Transparency for developers only works if it is also properly bounded; seeing your own usage should not mean seeing your colleague's.

5 IMPLEMENTATION

This section answers SQ3. The application is called Copilot Usage Web, with an Angular 20 frontend, a .NET 9 Minimal API backend, and SQL Server underneath. The part described here is the cost breakdown, which is my contribution. A second intern worked on usage-metrics features within the same codebase, so this section stays focused on the cost side.

5.1 From Manual Imports to a Live Database

The first version, built in April and May 2026, assumed an administrator would feed the system by hand. A manager would download the billing export from the GitHub portal and upload it through an import page in the app. I implemented a `CsvParsingService` for the CSV format, added NDJSON support when GitHub introduced a newer export format, and tracked uploads in an `ImportFile` table. This version was merged to master on May 26, 2026.

It worked, but it kept the same fundamental weakness as the spreadsheet process it replaced. Data was only as fresh as the last upload, and the upload still depended on one person. The interface was better, but the bottleneck was the same.

Two things changed that. The billing model switch on June 1, 2026 made costs vary per user, per model, and per day, so a monthly manual upload was no longer enough for anyone trying to track spending in any detail. At the same time, the main stakeholder decided to invest more in the project and set up an automated job that pulls billing data from GitHub and organisational data from internal company systems directly into the project's own SQL Server database. It runs on a schedule and requires no manual action.

This also fixed a problem with the old import approach that only became obvious later. The Copilot usage metrics API only exposes a rolling 28-day window of data. Under the manual import setup, if the person responsible forgot to run an export, or if a month went by without an upload, that data was simply gone. There was no

way to reconstruct it afterwards. With the automated job writing to the database continuously, that window no longer matters, as the data is stored before it ages out.

The switch landed in the codebase on June 9, 2026. The `CsvParsingService` was deleted, the import button was removed from the frontend, and the flat `CopilotUsageRecord` entity was reworked into a new `AiUsage` entity that matched the new database structure. A `TeamMember` entity was added for the organisational data, and `CostService` was rewritten to join the two tables at query time. The `ExceedsQuota` flag from the seat-based era was dropped. Previously the UI highlighted a user's usage in red once `ExceedsQuota` was true. Now the condition is the net amount: any row where the additional cost is greater than zero gets the red highlight.

5.2 Infrastructure and Secure Deployment

Deploying a dashboard that handles employee billing data inside a financial institution is a different problem than deploying a public web application. Van Lanschot Kempen operates under strict data governance requirements, and the data the application surfaces is tied to identifiable people. The infrastructure had to reflect that: all data stays inside the company's own Azure environment, no external service can reach the API, and no credential can be extracted from the pipeline or the container image. I designed and built the full deployment infrastructure for both the frontend and the backend from scratch, as part of the SQ3 implementation work.

The design follows a defence-in-depth approach, applying security controls at each layer independently: supply chain, container, secrets, and network. If one layer is bypassed, the others still hold. The NIST Zero Trust Architecture framework [15] makes the underlying principle explicit: no implicit trust is granted to anything based on its network location, so access has to be verified at every boundary rather than at a single perimeter. The layers themselves target risk categories that the OWASP Top 10 [17] ranks among the most common in web applications, including software supply chain failures and security misconfiguration.

5.2.1 Branching Strategy and Environment Separation.

Both pipelines run in Azure DevOps and follow the same branching model: a pull request targeting master triggers a build and deploys to a test environment (TST); a merge to master deploys to production (PROD). This gave every change I made a real deployed preview before it could touch production. Bugs that only manifest in a deployed context, such as environment variable mismatches or network policy conflicts, were caught in TST rather than discovered in production by a user.

Each pipeline is split across three YAML template files: orchestration, build, and deploy. This separation means the build template can be run and verified independently of the deploy target, which matters when debugging a failing image scan or a Bicep validation error in isolation.

5.2.2 Frontend Supply Chain Security.

The frontend build runs on a self-hosted Azure DevOps agent inside the company network, so the build process itself never leaves the perimeter. Node 24 is installed at build time and dependencies are installed with `npm ci` against the lockfile, which guarantees a

reproducible and tamper-evident install. Any deviation from the lockfile causes the install to fail.

Before any compilation happens, `npm audit` runs with `-audit-level=high -omit=dev`. This is a hard gate: a known high-severity vulnerability in a production dependency stops the pipeline. Using `-omit=dev` scopes the check to dependencies that actually reach the deployed artefact, avoiding false positives from build tooling. Supply chain attacks through compromised npm packages are one of the more realistic threats for a Node-based project [17], so catching them before compilation rather than after deployment is the right place for this check.

The deployment token for Azure Static Web Apps is retrieved at runtime from the Azure CLI rather than stored in pipeline variables. Tokens in variable groups can be read by any pipeline with access to that group and can appear in build logs if someone accidentally echoes them. Fetching at runtime eliminates both risks.

5.2.3 Backend: Infrastructure Validation Before Code.

The backend pipeline manages both application code and cloud infrastructure, and the order of operations matters.

The first step is not a build. It is an infrastructure validation. All Azure resources are defined as Bicep templates stored in the repository alongside the code. The pipeline runs `az bicep build` for syntax checking, then submits an ARM dry-run deployment against the real Azure subscription. This validates that the template is correct and that the subscription has the required resource providers registered, before a single line of application code is compiled. Catching infrastructure errors early avoids the situation where a correctly built image fails to deploy because of a misconfigured template.

After infrastructure validation, the .NET API is compiled in Release mode. The Docker image is then built with `-pull -no-cache`, so the base layer is always pulled fresh and no stale cached layer from a previous run can quietly introduce a vulnerability.

5.2.4 Container Image Security.

Before the image is pushed to the registry, an Orca container scan checks it for known CVEs across all image layers and for secrets accidentally embedded in the image. The scan runs with `continueOnError: false`: a finding that meets the severity threshold blocks the release entirely. The image is only pushed to the company's private Azure Container Registry after the scan passes clean. The registry is private, meaning no unauthenticated pull is possible from outside the company's Azure environment.

Images are tagged with the build number and branch name, so any running container can be traced back to the exact pipeline run and source commit that produced it.

The Dockerfile uses a four-stage multi-stage build, following Docker's own security guidance [4]. Stage one sets up the Alpine Linux runtime and creates a non-root user. Stage two installs the .NET SDK and restores packages. Stage three compiles and publishes the application. Stage four copies only the published binary into the stage-one base. The SDK, source files, and build cache never appear in the final image. Alpine Linux is used as the base because its minimal package set reduces the attack surface compared to a full distribution.

The container runs as the non-root user created in stage one, following Docker's own guidance to switch to an unprivileged user whenever a service does not need root [4]. A process running as root inside a container has more capability on the host if a container escape ever succeeds, so running unprivileged limits that blast radius. A HEALTHCHECK instruction is also included so the Azure Container Apps platform can detect and automatically restart unhealthy instances without manual intervention.

5.2.5 Network Isolation and Zero-Trust Identity.

The most important infrastructure decision was keeping the entire stack inside the company's virtual network. The Container App is bound to a shared Container App Environment that sits inside the company's Azure Virtual Network. Traffic between the frontend, the API, and the database never leaves that network boundary. External ingress is limited to HTTPS only; the API does not accept plain HTTP. This matches the principle that internal services should not be reachable from the public internet even if their credentials were somehow compromised [15].

Credentials for Azure service-to-service communication, including the pipeline pulling images from the Container Registry and the API authenticating to other Azure resources, use user-assigned Managed Identities rather than stored secrets [14]. A Managed Identity is an Azure AD identity tied to the resource itself. There is no password, no connection string, and no API key to rotate, leak, or accidentally commit to version control. The Azure platform resolves the identity at runtime and issues short-lived tokens automatically. One identity is assigned to the pipeline agent for ACR access; a separate identity is assigned to the running Container App for its own resource access. This separation of identities follows the principle of least privilege: each component can authenticate only to the services it needs.

Environment-specific values, the Key Vault URI, the database connection string, and the allowed CORS origins, are injected at deploy time as Bicep parameters rather than baked into the image. Separate parameter files (`tst.bicepparam` and `prod.bicepparam`) supply the values for each environment. The same container image is promoted from TST to PROD unchanged, with only the parameters differing. This means there is no risk of a TST configuration accidentally reaching production, and no need to rebuild the image for each environment.

5.3 Backend

The API exposes a single endpoint, `GET /api/costs`, registered through .NET 9's Minimal API pattern. Its query DTO carries four optional filter parameters: a partial username match, an exact model name, and a start and end date. All filtering happens inside the Entity Framework Core query before the results leave the database, so the response always contains only the records the caller asked for.

The business logic sits in `CostService` behind an `ICostService` interface. The service builds a LINQ query that joins the two tables on the shared license identifier, applies whichever filters are present, and projects the joined rows into a flat response DTO. The join is where department and team get attached to each billing row: the `AIUsage` table knows only the license identifier, and `TeamMember` supplies the human-readable name, department, and team. Each

field in the response DTO maps to exactly one column from one of the two tables, which keeps the projection explicit and easy to follow.

Dates are serialised as "yyyy-MM-dd" strings for safe cross-platform parsing.

5.4 Frontend Architecture

The frontend is built around Angular signals throughout. Data flows through a chain of computed signals inside a singleton `CostBreakdownService`: the HTTP response becomes `allRecords`, the active filter produces `filteredRecords`, the active grouping order produces `groupedRecords`, and the table flattens that into a `visibleRows` array for rendering. Every downstream signal recomputes automatically when anything upstream changes. Adjusting a filter updates the table and all three charts without any manual trigger, subscription, or re-fetch.

Data fetching uses `httpResource` from Angular 20, which integrates the HTTP call directly into the signal graph and exposes `isLoading` and `error` as signals automatically. During development a mock flag redirects the resource to a local JSON file, so frontend work does not require a live connection to the company database.

All grouping and filtering run in the browser. The full flat record list is fetched once and everything else happens client-side, which keeps the API simple and makes switching between grouping levels or changing a filter instant. For the current data volume this works well. If record counts grew by orders of magnitude, the grouping logic would need to move to the server, and I consider that an accepted trade-off for now.

All three components, the parent `CostBreakdown`, the `CostBreakdownTable`, and `CostBreakdownCharts`, inject the same service and read from its signals. There is no prop drilling and no duplicated state. Components use `OnPush` change detection, which means Angular only checks a component when one of its signal inputs actually changes.

5.5 The Cost Breakdown Table

Figure 2 shows the table with multiple levels expanded. A recursive function walks the filtered records level by level and builds a tree, following whatever grouping order is currently active. Each node in that tree carries totals for credits, gross, and net, computed once at build time.

Angular templates cannot render recursive structures natively, so the tree is flattened into one array of typed rows, each carrying its depth and a composite key. A single `@for` loop renders the whole table. CSS classes from `level0` to `level4` translate depth into indentation, font size, and background colour. Flattening was chosen over recursive child components because a flat array is much easier to paginate and keeps well under `OnPush` change detection.

Expand and collapse state is a signal holding a `Set` of composite keys, updated immutably so change detection notices each update. Pagination operates on top-level groups, fifteen per page, with ellipsis markers in the page list when the count is large. The column header row uses position: `sticky` so labels stay visible while scrolling.

5.6 Filter and Group-By Dialogs

The filter dialog (Figure 3) covers the three filter fields from the design: a partial username, a model, and a date range. The group-by dialog (Figure 4) lets the user activate grouping levels and reorder them, which is what makes the department-first versus model-first analysis from the design section possible.

Both dialogs use the same reversible edit pattern. Opening a dialog copies the current service state into a local temporary object. The dialog binds to that copy. Applying writes it back to the service signal in one step; cancelling throws the copy away. A half-finished edit never leaks into the live view.

5.7 The Charts

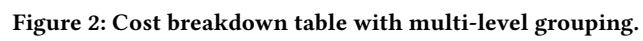
The chart panel implements the three visualisations from the design, built with `Chart.js` [3], an open-source charting library that renders on the HTML5 canvas. Each chart reads from `allRecords` directly and has its own independent filter controls, so adjusting a chart filter does not affect the table or the other charts.

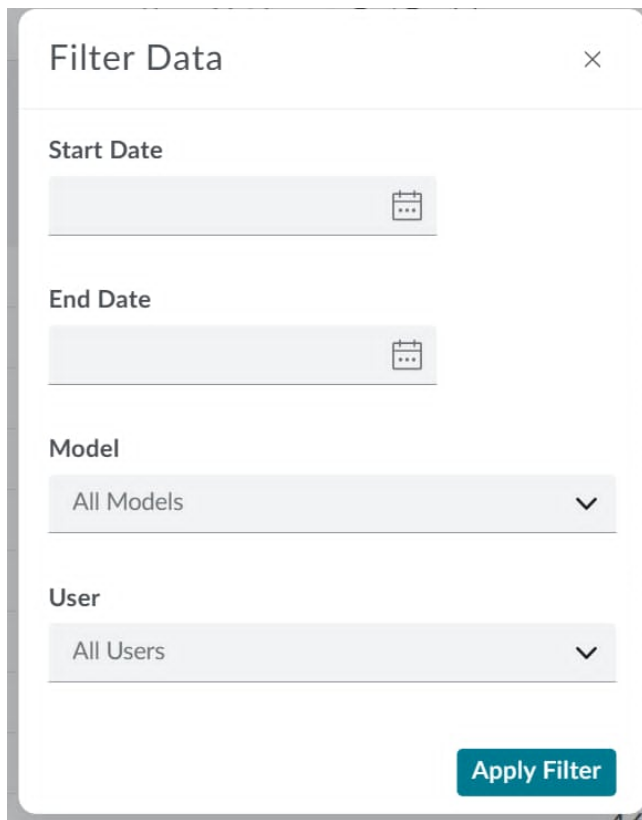
The first chart (Figure 5) is a vertical bar chart of credit consumption per AI model, filterable by user and time period. It answers the most common management question: which model is the expensive one.

The second chart (Figure 6) is a dual-axis line chart over time, switchable between daily and monthly granularity. Credits are on the left axis, net and gross cost on the right, which keeps consumption and money visually linked without one scale overwhelming the other. It can be narrowed to a single model or user.

The third chart (Figure 7) is a horizontal bar chart ranking users by gross cost, with a configurable top-N limit and optional model and time filters. This is the view that management asked about most in sprint reviews: who are the heaviest consumers, and has that changed.

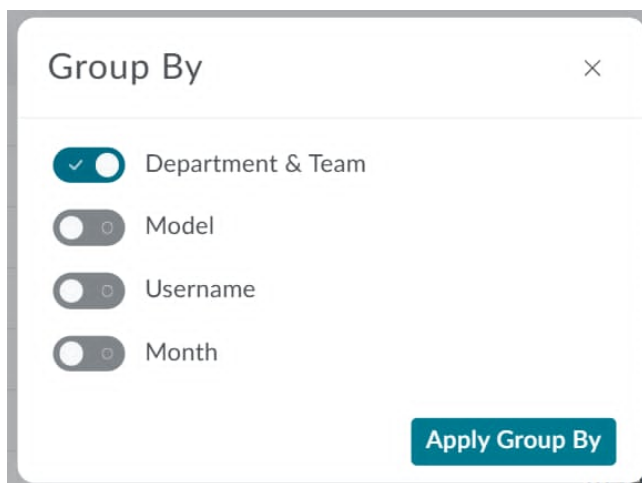
Each chart's dataset is its own computed signal, so it only recomputes when its specific inputs change. One practical detail: `Chart.js` renders onto a canvas element and its axis and tooltip callbacks sit outside Angular's rendering pipeline, so they cannot use Angular's built-in formatting pipes. All number formatting goes through a shared `formatNumber` method on the service, which wraps Angular's own `formatNumber` from `@angular/common` using the injected `LOCALE_ID`. The locale is set once to Dutch in the application config, and no locale string is hardcoded anywhere else in the codebase.





The 'Filter Data' dialog box is a light gray rounded rectangle with a close button (X) in the top right corner. It contains four filter sections: 'Start Date' and 'End Date' each with a light gray input field and a calendar icon; 'Model' and 'User' each with a light gray dropdown menu showing 'All Models' and 'All Users' respectively. A teal 'Apply Filter' button is at the bottom right.

Figure 3: Filter dialog.



The 'Group By' dialog box is a light gray rounded rectangle with a close button (X) in the top right corner. It contains four toggle switches: 'Department & Team' (checked), 'Model', 'Username', and 'Month' (all unchecked). A teal 'Apply Group By' button is at the bottom right.

Figure 4: Group-by dialog.

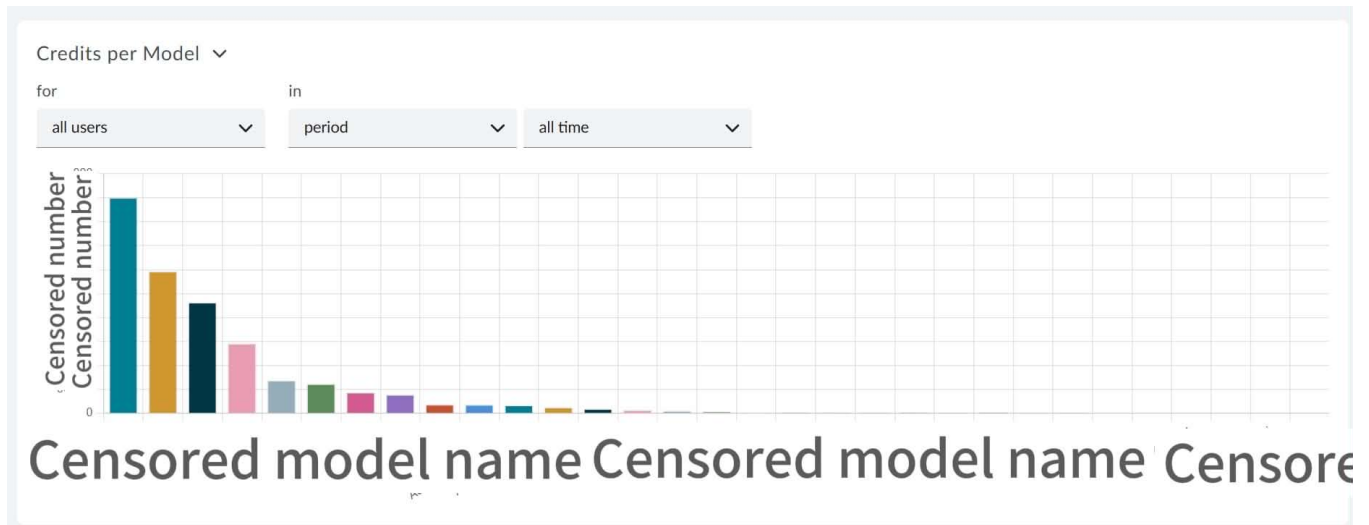


Figure 5: Credits per AI model.

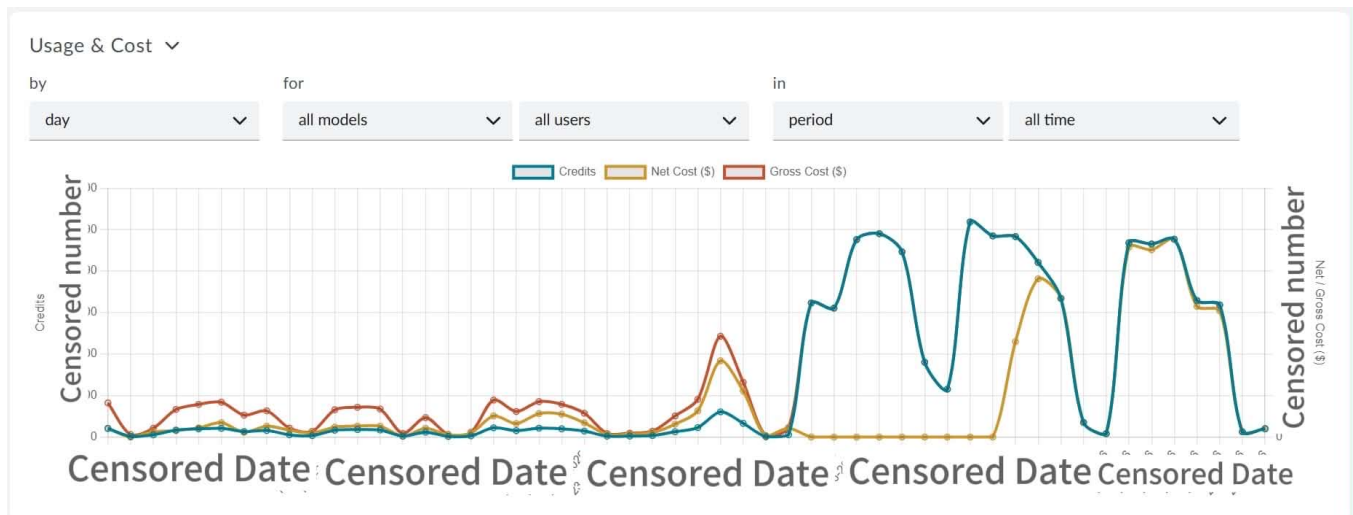


Figure 6: Credits and cost over time, dual axis.



Figure 7: Top users by gross cost.

6 EVALUATION

This section answers SQ4: to what extent the dashboard improves the interpretability and transparency of AI coding assistant utilization. The evaluation has two levels, following the treatment validation phase from Section 3. The first is continuous feedback gathered during development through sprint reviews. The second is a set of final interviews with three people from different roles, held at the end of the project.

6.1 Sprint Reviews

Development ran in two-week sprints, and each sprint ended with a review meeting. I presented the progress, showed the current state of the features, and the stakeholders gave feedback on the spot. These meetings were useful for evaluation mainly because of who attended. Both software engineers and a manager from the management team were there, so the same feature got looked at from the technical side and the management side in one go.

The feedback was mostly positive. Stakeholders were generally happy with the features as they were built. The more valuable input came in the form of concrete, use-case driven comments. In one review the naming of some fields was found confusing and was changed afterwards. In another, stakeholders asked for additional numbers to be shown in the charts and the table, since the figures they cared about were not all visible yet. This kind of feedback fed directly back into the next sprint, which is the feedback loop described in the engineering cycle in Section 3.

Two stakeholders were involved more closely than the sprint cadence. The main stakeholder, an expert software engineer who set up and maintained the database and handled GitHub plan changes including the move to token-based usage, was available for daily questions. The deployment infrastructure and security decisions, such as the network isolation approach and how service authentication is handled, were validated through those ongoing conversations rather than in the formal sprint reviews. The mentor, a senior software engineer, reviewed the code and checked its quality throughout. Their input shaped the design continuously rather than at fixed points.

6.2 Final Interviews

To get a more considered judgement than the running sprint feedback, the project ended with three face-to-face interviews, one per role. The aim was to hear from people who represent the dashboard's intended users and to cover different perspectives:

- a software engineer who was not involved in building the dashboard, to get an outside developer's view
- the main stakeholder, who knows the data and the billing context in depth
- a manager from the management team, who represents the management side of the audience

The interviews were semi-structured. Each person got a few questions tailored to their role, followed by one or two open questions so they could raise things the fixed questions miss. They were kept short on purpose, to respect the participants' time and keep the answers focused.

6.2.1 Interview Questions.

Software engineer (outside the project):

- When you open the dashboard for the first time, can you find your own usage and understand it without help?
- Compared to having no view of your usage at all, does this give you information you find useful?
- Is anything in the charts or table unclear or misleading?

Main stakeholder:

- Does the dashboard make the usage and cost data easier to explore than the previous spreadsheet exports?
- Are the dimensions you can filter and group by the right ones for the questions you usually have?
- Is there anything important about the data that the dashboard still cannot show?

Manager (management team):

- Can you answer cost questions yourself through the dashboard, without asking someone to prepare a report?
- Does the way costs are broken down by team, model, and time match how you think about the budget?
- Would you trust this view enough to use it for decisions about Copilot usage?

Open questions (all participants):

- What would you change or add to make the dashboard more useful for you?
- Did using the dashboard change how transparent the Copilot usage feels to you?

6.3 Results

6.3.1 Software Engineer (Outside the Project).

Jan Richter was able to locate his own data in the dashboard without help, though it took some time because of the number of options. He was already a partial exception: he had found a VS Code extension that shows credit consumption per interaction, so he had some sense of his usage. What the dashboard added was the money side, how much those credits cost, and how his usage sits relative to the team. His main criticism was about the charts: too many colours, no obvious search bar, and no ranking indicators. He also raised a point about GitHub Copilot itself showing only credits to developers in its interface, even though the underlying cost data exists. The dashboard does show costs, so that confusion is partly resolved here.

6.3.2 Main Stakeholder.

Nicolas confirmed the dashboard is significantly better than the previous process, which required downloading three separate files from the GitHub portal and combining them manually in Excel each time someone had a question. The automated pipeline removes that entirely. He said the filter and grouping options match the questions managers and team members actually ask, and noted that since the new billing model launched, developers have been asking about their costs because GitHub does not show per-user spend clearly. The dashboard fills that gap. He raised two missing features: an export function for users who want to work with the data outside the dashboard, and a bottom-ten users chart to complement the existing top-N view.

6.3.3 Manager (Management Team).

The manager confirmed they could answer cost questions directly in the dashboard, without asking anyone to prepare a report. On whether the breakdown by team, model, and time matches how the management team thinks about the budget, the answer was almost. What is missing is budget context. The dashboard shows what is being spent, but not out of how much, so a number on its own does not say whether a team is within its budget or past it.

The manager also pointed at an ambiguity in the credit figures themselves. When a model shows high credit consumption, the view does not tell you whether that model is simply used a lot, or whether it burns more credits per interaction even when used less. Answering that would require usage counts next to the cost figures, which is exactly the data held by the usage-metrics side of the project that the cost view is not yet connected to.

On trust, the answer was a clear yes: the manager would use this view for decisions about Copilot usage. The suggested improvements were small, tooltips to make the interface friendlier and a more intuitive filtering and grouping UI. Asked whether the dashboard changed how transparent Copilot usage feels, the answer was “definitely yes”.

6.3.4 Summary.

All three interviews point in the same direction. Developers get a view of their own cost and usage that did not exist before. The stakeholder responsible for cost oversight can now answer questions across teams and models without a manual export. The manager can answer budget questions independently and trusts the view enough to base decisions on it. The friction points raised were chart and filtering usability details, a missing export feature, the absence of budget context, and the credit-based billing unit from GitHub, which hides whether high credit numbers come from heavy use or from expensive models. None of those undermine the core finding: the dashboard improves both interpretability and transparency of AI coding assistant utilisation, which is what SQ4 asks.

7 DISCUSSION

7.1 Answers to the Research Questions

SQ1: How can usage logs from AI coding assistants be structured into an analytical data model?

The raw billing export from GitHub gives one row per user per day per model, with three cost fields: credits consumed, gross amount, and net amount. That is already a usable fact table. The harder part was organisational data. A GitHub billing row carries only a licence identifier, not a name, team, or department. Storing those fields directly on the billing row would make them stale every time someone moves teams. Instead, a second table maps licence identifiers to display names and organisational placement, and the two are joined at query time. That separation keeps billing history clean and means the frontend only ever sees human-readable names. The five grouping dimensions that resulted from this, user, model, date, department, and team, cover every question that came up during stakeholder interviews.

SQ2: What visualization techniques are most appropriate for analyzing AI assistant usage logs?

The chart types were chosen from the questions users actually ask, not from a general catalogue. A vertical bar chart answers which model costs the most. A dual-axis line chart shows how usage and cost move over time without one scale drowning the other. A horizontal bar chart ranks users by cost. The table covers exact figures and organisational drill-down, where charts fall short. Keim et al. [13] argue that interactivity is what separates visual analytics from static reporting, and filtering and grouping were treated as requirements from the start rather than additions.

SQ3: How can a web-based dashboard be implemented and securely deployed to support analysis of AI coding assistant usage inside a corporate environment?

The application was built with an Angular 20 frontend, a .NET 9 Minimal API backend, and SQL Server. The frontend uses Angular signals so every filter or grouping change updates all views instantly without a re-fetch. The backend exposes a single endpoint that joins the two tables and applies server-side filters, keeping the API simple. The switch from manual file imports to a direct database connection in June 2026 removed the last manual step in the data pipeline.

Deploying inside a financial institution added a layer of complexity that is not part of a typical university project. The application handles billing data tied to identifiable employees, so all infrastructure had to run inside the company’s Azure virtual network with no external access to the API. I built the full CI/CD pipeline from scratch, covering supply chain security for the frontend (npm audit as a hard gate), container image scanning with Orca before any push to the registry, infrastructure-as-code validation via Bicep dry-run before compilation, and credential-free service-to-service authentication through Azure Managed Identities. The container itself runs as a non-root user on a minimal Alpine Linux image so that the attack surface of what actually ships to production is as small as possible.

SQ4: To what extent does a web-based dashboard improve the interpretability and transparency of AI coding assistant utilization?

Sprint reviews throughout development showed that stakeholders from both the technical and management side could read and engage with the features as they were built. The final interviews confirmed the improvement more directly. The main stakeholder described the previous process as downloading three separate files and combining them in Excel every time someone had a question. The dashboard replaced that entirely. A developer who had no prior visibility into his own cost could locate his data and see what his usage translated to in money. The manager could answer cost questions without asking anyone for a report and said they would trust the view for decisions about Copilot usage. All three said transparency improved. The remaining friction points were usability details in the charts and filters, a missing export feature, and the lack of budget context next to the spend figures. None of these change the core outcome.

7.2 Contributions

On the practical side, the two-table data model, the automated pipeline, and the role-based access design are reusable decisions for any team in a similar position after GitHub’s billing change. Other organisations adopting credit-based AI tooling face the same

problem: the data exists, but there is no standard way to make it explorable. The architecture here is one worked answer to that.

The academic contribution is narrower but concrete. Peng et al. [18] study Copilot in a controlled experiment and Brandebusemeyer et al. [2] in a four-day field study; neither says much about what continuous, organisation-wide observation looks like. This thesis fills that gap with a real deployment, real data, and the design decisions that came from it.

7.3 Limitations

The evaluation covers one organisation, one team, and three interviews, one per role. The usability and transparency conclusions are directionally clear, but a stronger claim would need more participants across more varied settings.

Within the application itself, the cost breakdown and the usage-metrics views built by a second intern sit in the same codebase but are not connected. The cost data and the behavioural data are separate, which means a question like “which team spends the most but gets the least out of it” cannot yet be answered in one view. The manager interview surfaced a concrete case of this: a high credit number for a model does not reveal whether that model is used often or is expensive per interaction, and separating the two needs usage counts next to the cost figures.

Role-based access control is planned as the immediate next development step. Once a developer can sign in and see only their own data, the individual transparency benefit extends to everyone, not just those with admin access.

7.4 Future Work

Finishing authentication is the most pressing next step, not as a feature but as a prerequisite. The transparency argument for individual developers only holds once they can sign in and see their own data without depending on anyone else. Until then that part of the value proposition remains undelivered.

Linking the cost view to the usage-metrics view is where the more interesting analytical questions live. Brandebusemeyer et al. [2] showed that the type and intensity of interaction matters for productivity, not just whether someone uses Copilot. A view that surfaces cost alongside behavioural patterns would let a manager ask whether the spending is concentrated in interactions that actually move work forward, or spread across low-value ones. That question cannot be answered with cost data alone.

Smaller additions came directly from the interviews: an export function, a chart showing the bottom-N users by consumption, tooltips and a more intuitive filtering UI, and showing spend against the allocated budget instead of as a bare number. All of these fit within the current architecture without significant rework.

8 RELATED WORK

Most research on AI coding assistants looks at whether they make developers more productive, not at how an organisation can analyze its own usage of them. That difference is what positions this thesis.

8.1 Productivity Studies

The most cited evidence for the productivity benefit comes from Peng et al. [18]. They ran a randomized controlled experiment with

95 professional developers implementing an HTTP server, and the group with Copilot finished 55.8% faster. The gains were uneven: less experienced developers and those who code more hours per day benefited the most. One detail is relevant here. When asked to estimate their own improvement, participants guessed around 35%, far below the measured 55.8%. Developers are not reliable judges of their own usage. That gap between perception and reality is precisely the kind of thing that only shows up when you look at the data.

Peng et al. also note their study uses a standardised single-person task and does not examine real-world collaborative settings. That is a limitation they acknowledge, and it leaves open the question of what Copilot usage actually looks like across a whole engineering team over time.

Brandebusemeyer et al. [2] get closer to that. In a four-day mixed-methods study with 22 SAP developers, they found that using only chat or only in-code suggestions made developers significantly faster, but combining both interaction types in one task brought no measurable benefit. Intensity mattered too: in-code suggestions kept paying off up to around thirteen interactions per task and chat up to six, after which durations exceeded the no-Copilot baseline. During the uncontrolled workdays, 120 of 445 tasks (27.42%) involved AI, and developers reported feeling more productive on those tasks while also reporting higher cognitive load.

That study connects to this thesis in two ways. Brandebusemeyer et al. distinguish between three Copilot interaction types, in-code suggestions at the cursor, in-line chat, and the chat window, and note that each runs through a different prompt template and differs in how much of the codebase it considers as context. Their paper does not discuss cost, but the connection is direct: under credit-based billing the cost of an interaction depends on the model and the number of tokens consumed [12], and an interaction type that pulls in a wider context feeds the model more input. They also conclude that how and how much developers interact with Copilot matters more than whether they use it at all. An organisation cannot know which pattern it has without access to per-user, per-model usage data over time. That data exists in the billing logs but has not been made accessible or interactive before.

8.2 Visual Analytics

Keim et al. [13] define visual analytics as combining automated analysis with interactive visualisation for reasoning over large and complex datasets. Their core argument is that fully automated analysis lacks communicability: if a result turns out to be wrong, there is no way to examine the process that produced it. Interactive visualisation closes that gap by keeping the human in the loop. They describe the iterative process as a sense-making loop, where a user alternates between looking at the data and steering what they look at next.

They also identify the tension between overview and detail. A visualisation that shows only the aggregate hides specifics; one that shows only specifics makes patterns invisible. Keim adjusts Shneiderman’s well-known “overview first” mantra for visual analytics into: analyse first, show the important, zoom, filter and analyse further, details on demand. That maps directly onto the design used

here: the charts show the important patterns up front, the collapsible table gives the drill-down and exact numbers on demand, and both share the same filter state.

Keim et al. specifically list financial analysis as a prototypical application for visual analytics, describing the need to integrate heterogeneous data streams at high update rates into a unified task-centred view. The Copilot billing context fits that description: billing records, organisational data, and model usage all arrive from different sources and need to be joined before they make sense. Their work is conceptual and predates AI coding assistants entirely. Applying those principles to this specific dataset and the organisational questions it needs to answer is what this thesis contributes.

8.3 Secure Deployment in Regulated Environments

Software that processes employee data inside a financial institution does not just need to work; it needs to meet legal and technical requirements before it can run. The EU Digital Operational Resilience Act [5], which became applicable in January 2025, requires financial entities to maintain an ICT risk management framework that covers the full software lifecycle, including controls over third-party software components and audit rights over the supply chain. Van Lanschot Kempen, as a Dutch financial institution, operates under this regulation.

The NIST Secure Software Development Framework [16] describes what secure delivery looks like in practice. It defines four groups of practices: prepare the organisation, protect the software, produce well-secured software, and respond to vulnerabilities. Relevant to this thesis, it treats software supply chain protection and the verification of third-party components as part of secure development, not as separate concerns. These are not optional additions for high-security projects; they are baseline expectations for any software that touches personal or financial data.

Neither DORA nor the SSDF tells you how to build an analytics dashboard for AI tool usage. What they define is the floor a deployment must meet. That context shapes what "implementing" the dashboard means in practice, as SQ3 reflects.

8.4 What Is Missing

None of the above papers build something an organisation can actually run. Peng et al. measure a controlled task. Brandebusemeyer et al. observe developers at one company for four days with significant instrumentation. Keim et al. describe principles without applying them to a specific domain. And none of them address what the preceding subsection establishes: that shipping software into a regulated financial institution is itself a defined problem, with legal obligations around supply chain security and ICT risk that shape every deployment decision. What is missing is a persistent system that collects Copilot usage data as it is generated, structures it into an analytical model that covers both cost and usage dimensions, and presents it through a dashboard that both managers and developers can explore on their own, built to a standard that lets it actually run in a corporate environment. That is what this thesis builds.

9 CONCLUSION

This thesis set out to answer how the utilization of AI coding assistants can be systematically analyzed through a web-based analytics dashboard. The problem at Van Lanschot Kempen was not a lack of data. The usage data existed, but it sat behind a manual export process that one person controlled, so management saw only monthly totals and developers saw nothing about their own usage. GitHub's move to token-based billing made that situation harder to live with, since costs now vary per user, per model, and per day depending on actual consumption.

The first concrete decision was the data model. A GitHub billing row carries only a licence identifier, not a name or an organisational placement. Storing department and team directly on each billing row would make that data stale every time someone moved. The solution was two tables joined at query time: one for the billing facts, one for the organisational metadata. That separation keeps history clean and gives the five dimensions, user, model, date, team, and department, that every stakeholder question turned out to need.

The visualisation choices followed from the questions, not from a catalogue. Which model costs the most is a bar chart. How usage changes over time is a line chart with two y-axes because credits and money are different scales. Who the heaviest users are is a horizontal bar. For exact numbers and departmental drill-down, a collapsible table does what no chart can. These were built into a working application, Copilot Usage Web, with an Angular 20 frontend, a .NET 9 Minimal API backend, and a direct database connection. Because the application runs inside a financial institution bound by the EU Digital Operational Resilience Act [5] and handles employee billing data, it also required a secure deployment infrastructure: CI/CD pipelines with dependency vulnerability scanning, container image scanning before any push to the registry, infrastructure-as-code validation, and a network configuration that keeps the API entirely inside the company's Azure virtual network with credential-free service authentication through Managed Identities.

The evaluation addressed SQ4. Sprint reviews showed both technical and management stakeholders could read and engage with the features as they were built. The final interviews gave more direct evidence. The main stakeholder described the previous process as downloading three separate files from GitHub and combining them in Excel each time someone had a question. The dashboard replaced that entirely. A developer who previously had no view of his own cost could find his data and see what his usage translated to in money. The manager from the management team could answer cost questions without asking anyone for a report and would trust the view for decisions about Copilot usage. Asked whether transparency improved, all three said yes, the manager with a "definitely". The friction points raised, chart and filter usability details, a missing export feature, and the lack of budget context, are implementable improvements that do not change that finding.

Several things remain. Authentication and role-based access control are fully designed and planned as the next development step, after which the dashboard can be opened to all employees. Once a developer can sign in and see only their own data, the individual transparency benefit becomes real rather than something that depends on someone else setting up the view.

The cost breakdown built here and the usage-metrics features built alongside it by a second intern are not yet connected. Linking them is the most valuable next step. Knowing what Copilot costs per user per model is useful. Knowing whether that cost comes from productive daily use or from a small number of expensive interactions is more useful. The manager interview gave a concrete version of this question: a high credit number alone does not say whether a model is used heavily or simply burns more credits per interaction. Brandebusemeyer et al. [2] found that the type and intensity of interaction affects productivity in ways that simple adoption numbers miss. A dashboard that brings cost and behavioral data together in one view would let an organisation start to answer not just how much it is spending, but whether it is spending it well.

Smaller additions that came directly from the interviews include an export function for users who want to take data out of the dashboard, a bottom-N users chart to complement the existing top-N view, and showing spend against the allocated budget so a cost figure carries its own context. All are straightforward to add within the current architecture.

REFERENCES

- [1] Ambilykkara. 2024. Copilot Metrics Retention - GitHub Marketplace Action. <https://github.com/marketplace/actions/copilot-metrics-retention>. "The Copilot Metrics API supplies data spanning a 28-day timeframe".
- [2] Charlotte Brandebusemeyer, Tobias Schimmer, and Bert Arnrich. 2026. Developers' Experience with Generative AI: First Insights from an Empirical Mixed-Methods Field Study. In *IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE SEIP)*. IEEE.
- [3] Chart.js Contributors. 2024. Chart.js - Open source HTML5 Charts. <https://www.chartjs.org/>.
- [4] Docker Inc. 2024. Building best practices. <https://docs.docker.com/build/building/best-practices/>.
- [5] European Parliament and Council of the European Union. 2022. Regulation (EU) 2022/2554 on Digital Operational Resilience for the Financial Sector (DORA). <https://eur-lex.europa.eu/eli/reg/2022/2554/oj/eng>. Applicable from January 17, 2025.
- [6] Minko Gechev. 2025. Announcing Angular v20. <https://blog.angular.dev/announcing-angular-v20-b5c9c06cf301>.
- [7] GetPanto. 2026. GitHub Copilot Statistics 2026 — Users, Revenue & Adoption. <https://www.getpanto.ai/blog/github-copilot-statistics>.
- [8] GitHub. 2026. Copilot ask, edit, and agent modes: What they do and when to use them. <https://github.blog/ai-and-ml/github-copilot/copilot-ask-edit-and-agent-modes-what-they-do-and-when-to-use-them/>.
- [9] GitHub. 2026. GitHub Copilot Features. <https://docs.github.com/en/copilot/get-started/features>.
- [10] GitHub. 2026. GitHub Copilot is moving to usage-based billing. <https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/>.
- [11] GitHub. 2026. GitHub Copilot Plans & Pricing. <https://github.com/features/copilot/plans>.
- [12] GitHub. 2026. Models and pricing for GitHub Copilot. <https://docs.github.com/en/copilot/reference/copilot-billing/models-and-pricing>.
- [13] Daniel Keim, Gennady Andrienko, Jean-Daniel Fekete, Carsten Görg, Jörn Kohlhammer, and Guy Melançon. 2008. Visual Analytics: Definition, Process, and Challenges. In *Information Visualization: Human-Centered Issues and Perspectives*. Lecture Notes in Computer Science, Vol. 4950. Springer.
- [14] Microsoft. 2024. What are managed identities for Azure resources? <https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>.
- [15] National Institute of Standards and Technology. 2020. Zero Trust Architecture (NIST SP 800-207). <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf>.
- [16] National Institute of Standards and Technology. 2022. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. Technical Report SP 800-218. National Institute of Standards and Technology.
- [17] OWASP Foundation. 2025. OWASP Top 10: 2025. <https://owasp.org/Top10/2025/>.
- [18] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590.
- [19] Van Lanschot Kempen. 2026. Digital & Technology - Van Lanschot Kempen Careers. <https://careers.vanlanschotkempen.com/en-nl/expertise/digital-and-technology>.
- [20] Roel J. Wieringa. 2014. *Design Science Methodology for Information Systems and Software Engineering*. Springer.

AI

A AI USAGE CONSIDERATIONS APPENDIX

The work in this thesis is mine. I scoped the problem with the stakeholders, built the application and the deployment pipeline, ran the interviews, and wrote up what came out of them. Where AI tools came in was the writing itself, so this appendix explains that use against the three criteria above.

Originality. LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the author to ensure accuracy and originality. Concretely: I drafted the text, then used Grammarly and LLM assistants to catch grammar mistakes and awkward phrasing. English is not my first language, so this mattered more for readability than for content. None of the numbers, citations, design decisions, or interview results came from an AI

tool. I checked each cited claim against the source paper or page myself before keeping it.

Transparency. There is some irony in that AI coding assistants are the topic of this thesis, but AI was not part of the method. No result in the evaluation and no decision in the design was generated by a model. One thing worth stating plainly: GitHub Copilot was available in the company development environment while I built the application, and I used it for code suggestions like every other developer at Van Lanschot Kempen does. The architecture and the decisions in Sections 4 and 5 are mine. When a writing tool suggested a formulation, I only kept it after checking it still matched what the application actually does.

Responsibility. I did not train any model or collect data for training, so those criteria do not really apply here. My AI use was short editorial interactions, nothing at the scale where the environmental footprint becomes a serious question.